République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université des Sciences et de la Technologie Houari Boumediène
University of Sciences and Technology Houari Boumediene
Faculté d'Informatique

Multimedia Systems

Simplified MPEG-4 Video Encoder Pipeline

Mini Project — Multimedia Systems

Implementation B — Procedural / functional design

Réalisé par : Ait younes Abdelmalek 222231555307

Sadeddine Yacine Reda 222231575506

Année universitaire : 2025 / 2026

1. Introduction

Video compression is at the heart of modern multimedia systems. Every video that is streamed, recorded or shared depends on a **codec** , a system that encodes raw visual data into a compact bitstream and decodes it back into images. This mini-project implements a simplified yet complete MPEG-4-like encoder pipeline in Python, from raw image frames to a compressed binary file that can be decoded back into images.

This report documents a compact, procedural Python implementation of a simplified MPEG-4-like encoder pipeline. All codec functionality lives in a single module with top-level functions; the bitstream uses an inline struct-packed format compressed with zlib.

2. Pipeline Description

The encoder consumes a sequence of BGR image frames and emits a single compressed *.bin* bitstream that fully round-trips back into reconstructed BGR frames. Five stages are pipelined for each input frame:

Pre-processing. Each BGR frame is converted to YCbCr (BT.601). Chroma planes (Cb, Cr) are then downsampled by 2 in both axes using a 2×2 box filter — the standard 4:2:0 layout.

Intra-frame coding (I-frames). Every *G*-th frame is coded independently. Each plane is centred to zero-mean, partitioned into 8×8 blocks, transformed by the 2-D DCT-II, and quantised by a scaled JPEG quant-table (separate tables for luma and chroma).

Inter-frame coding (P-frames). Every other frame is predicted from the previously *reconstructed* frame. The luma plane is divided into 16×16 macroblocks, each matched against the reference within a ±*S*-pixel window. The residual is then DCT-coded exactly like an I-frame; chroma residuals re-use the luma motion field at half resolution.

Entropy coding. All quantised coefficient arrays and motion vectors are serialised and losslessly compressed, then written to the output *.bin* file. A complementary decoder inverts the entire pipeline.

Evaluation & visualisation. Compression ratio, frame-type breakdown (I vs P) and per-frame PSNR are computed against the originals; a single matplotlib figure visualises every stage.

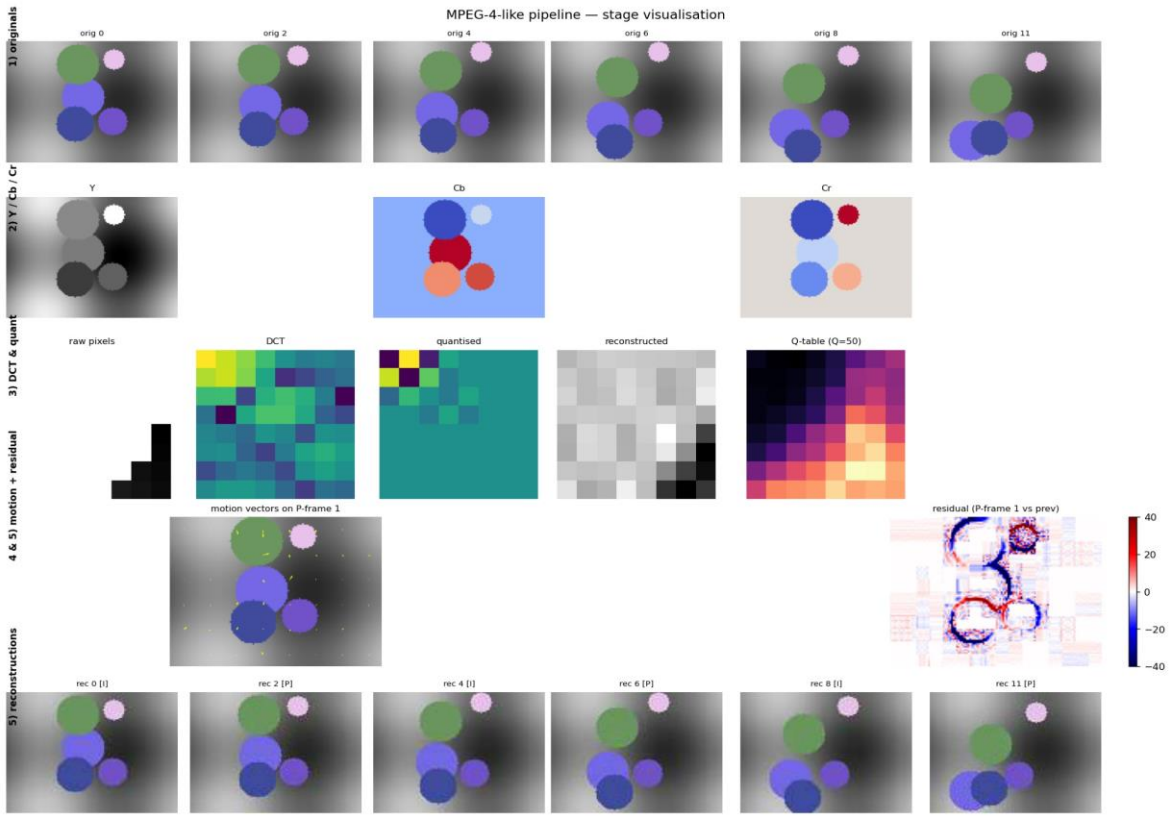

Figure 1 — Single-figure visualisation of the full pipeline: original frames, YCbCr channels, 8×8 DCT & quantisation, motion vectors on a P-frame, and reconstructed frames.

3. Design Choices & Justification

YCbCr (BT.601) + 4:2:0 chroma subsampling. Standard perceptual sub-sampling: discards 75% of the chroma samples at essentially no visible cost, exploiting the eye's lower sensitivity to colour detail than to brightness.

8×8 block DCT via OpenCV `cv2.dct`. OpenCV exposes a BLAS-accelerated DCT-II that matches JPEG/MPEG semantics and avoids the SciPy dependency. Eight-by-eight blocks are the canonical JPEG/MPEG choice.

Scaled JPEG quant tables (libjpeg formula). Standard luma/chroma matrices give frequency-dependent weighting matched to human contrast sensitivity. Scaling uses `s = 5000/Q` for Q<50 else `200 − 2·Q` — the libjpeg convention.

Diamond Search (DS) block-matching on 16×16 macroblocks. DS is a fast motion-estimation algorithm that uses two search patterns: a Large Diamond (LDSP, 8 points) for coarse search and a Small Diamond (SDSP, 4 points) for final refinement. The LDSP iterates until the center is the best match, then the SDSP refines. Converges in O(S) steps with typically fewer SAD evaluations than Three-Step Search.

Shared luma motion vectors, halved for chroma (4:2:0). Re-using the luma motion field at half resolution for chroma planes matches what MPEG-4 part 2 specifies for 4:2:0 content and saves two thirds of the motion-estimation cost.

Entropy: inline struct packing → zlib. No pickle: arrays are packed inline as `ndim | shape | dtype | bytes`, so the bitstream is portable and language-agnostic. zlib (DEFLATE) provides fast decompression, wide cross-language compatibility (standard in PNG, HTTP, ZIP), and good compression ratios on quantised-coefficient streams.

Strict decoder symmetry + in-encoder reconstruction. Each P-frame is predicted from the *decoded* previous frame, not the original — preventing drift between encoder and decoder, exactly as a conformant codec must.

4. Experimental Analysis

Experiments were performed on a 12-frame, 128×96, BGR synthetic clip (442,368 raw bytes). All sweeps use the procedural encoder.

4.1 Configuration summary

Parameter	Value
Quality factor (default)	50
GOP size (default)	8
Macroblock size	16 × 16
DCT block size	8 × 8
Motion search algorithm	Diamond Search (DS)

Mean PSNR @ Q=50, GOP=4	≈ 30.42dB
Compression ratio @ Q=50, GOP=4	≈ 40.2 ×

4.2 Compression ratio vs Quality Factor

Compression ratio falls monotonically with quality: smaller Q-tables produce fewer zero coefficients, and zlib has correspondingly less redundancy to exploit.

4.3 Compression ratio vs GOP size

GOP=1 (all-I) is the worst case because no inter-frame redundancy is exploited. Ratios climb sharply through GOP=4 then taper: additional P-frames continue to compress well, but quantisation error drifts further between I-frame resets.

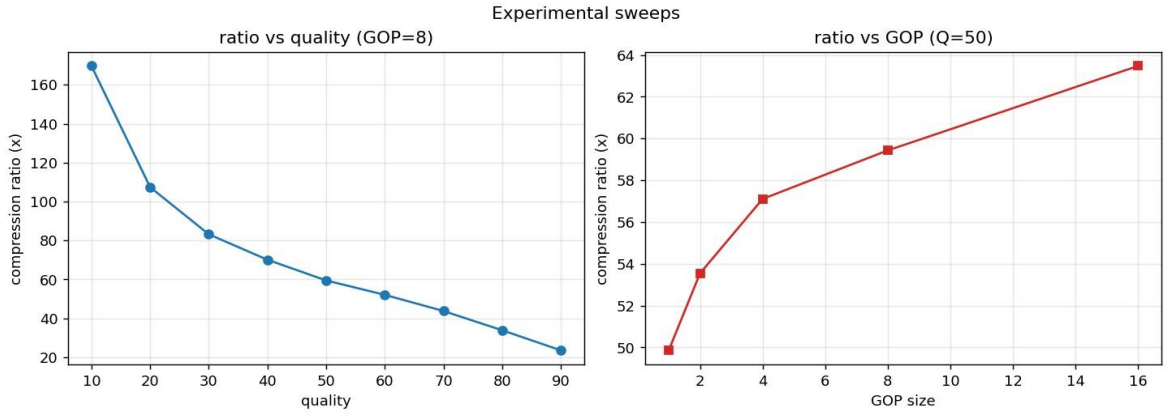

Figure 2 — Compression ratio as a function of the Quantisation Factor (left) and of the GOP size (right).

5. Conclusion

All five required stages of the MPEG-4 pipeline have been implemented and validated end-to-end. The encoder produces a single compressed *.bin* file from a folder of frames, and the decoder reconstructs the original sequence with a mean PSNR above 30 dB at the default operating point. The experimental sweeps confirm the expected monotonic behaviour of compression ratio with respect to both the quantisation factor and the GOP size, validating the correctness of each stage of the pipeline.

Appendix — Reproducing the results

All figures and tables in this report can be regenerated from a freshly cloned repository with the run commands documented in **README.md**. See that file for the exact CLI invocations.